

УНИВЕРЗИТЕТ У БЕОГРАДУ
МАТЕМАТИЧКИ ФАКУЛТЕТ



Павле Влајковић

ЕМПИРИЈСКО ПОРЕЂЕЊЕ
АРХИТЕКТУРАЛНИХ ОБРАЗАЦА
СОФТВЕРСКИХ СИСТЕМА НА ПРИМЕРУ
ОБРАДЕ МУЛТИМЕДИЈАЛНОГ
САДРЖАЈА

мастер рад

Београд, 2026.

Ментор:

др Владимир Филиповић, редовни професор

Чланови комисије:

др Александар Картељ, ванредни професор

др Сташа Вујичић Станковић, доцент

Датум одбране: _____

Мојој породици, са захвалношћу

Наслов мастер рада: Емпиријско поређење архитектуралних образаца софтверских система на примеру обраде мултимедијалног садржаја

Резиме: Избор између монолитне, модуларно монолитне и микросервисне архитектуре утиче на границе система, начин распоређивања и употребу ресурса, али опште препоруке не одређују понашање конкретне имплементације. Циљ рада је поређење ових архитектуралних облика под контролисаним условима на истом систему за обраду видео записа. Полазни монолит мигриран је најпре у модуларни монолит са израженим модулским границама, а затим у микросервисну варијанту са независно распоредивим пословним сервисима. Поређење обухвата монолит, модуларни монолит и микросервисне топологије са једном и две инстанце сервиса за транскодовање. Цело распоређивање је у свакој експерименталној варијанти ограничено заједничким буџетом процесора и меморије, а мерења су спроведена у два ресурсна профила, при секвенцијалном и паралелном подношењу захтева. Већи профил био је праћен већом пропусношћу у свим експерименталним варијантама. У односу на монолит, средња пропусност модуларног монолита кретала се од 9,3% ниже до 2,3% више, док су микросервисне топологије углавном имале нижу пропусност, дуже трајање и већу употребу меморије. Топологија са две инстанце сервиса за транскодовање није била доследно боља од топологије са једном када укупан капацитет није повећан. Практични допринос рада чине упоредив поступак мерења под заједничким ресурсним буџетом и следљив приказ двоетапне миграције. Налази су описни и ограничени на испитани систем, оптерећења и окружење, па не успостављају универзалан поредак архитектуралних образаца.

Кључне речи: софтверска архитектура, монолит, модуларни монолит, микросервиси, транскодовање видео записа, мерење перформанси, заједнички ресурсни буџет

Садржај

1	Увод	1
1.1	Предмет и мотивација рада	1
1.2	Циљ и истраживачка питања	2
1.3	Обим и приступ	3
1.4	Намераваани допринос и структура рада	3
2	Теоријске основе	5
2.1	Критеријуми класификације	5
2.2	Монолит	5
2.3	Модуларни монолит	6
2.4	Микросервисна архитектура	6
2.5	Синтеза критеријума	7
3	Методологија	9
3.1	Дизајн студије и систем под тестом	9
3.2	Експериментално окружење	11
3.3	Променљиве, оптерећење и ресурсни профили	11
3.4	Операционе дефиниције метрика	12
3.5	Извођење експеримента и обрада резултата	14
3.6	Квалитативно поређење и евиденција миграције	15
3.7	Синтеза, валидност и поновљивост	16
4	Монолитна архитектура	18
4.1	Организација и јавни програмски интерфејс	19
4.2	Пријем и чување видео записа	20
4.3	Асинхрона израда рендиције	20
4.4	Конкурентност и границе комуникације	21

4.5	Распоређивање и инструментација	22
5	Модуларни монолит	23
5.1	Модулске одговорности и објављени интерфејс	23
5.2	Зависности и провера модулских граница	24
5.3	Јавни програмски интерфејс и ток обраде	25
5.4	Комуникација и границе података	26
5.5	Распоређивање и инструментација	26
5.6	Прва етапа миграције	26
6	Микросервисна архитектура	28
6.1	Топологија и јединице распоређивања	28
6.2	Власништво над операцијама и подацима	29
6.3	Јавни програмски интерфејс	31
6.4	Ток порука и трансакционе границе	31
6.5	Типови порука и границе поузданости	31
6.6	Распоредна граница и инструментација	34
6.7	Друга етапа миграције	34
7	Резултати и дискусија	36
7.1	Пропусност и трајање оптерећења	36
7.2	Латенција успешне групе	38
7.3	Употреба процесора и меморије	40
7.4	Шта резултати показују	41
8	Закључак	43
8.1	Одговори на истраживачка питања	43
8.2	Практично значење и будући рад	44
	Библиографија	46

Глава 1

Увод

1.1 Предмет и мотивација рада

Софтверска архитектура описује основну организацију система, односе међу његовим деловима и одлуке које усмеравају даљи развој [3]. Избор између монолита, модуларног монолита и микросервисне архитектуре стога није избор назива или програмског оквира, већ избор граница унутар којих се код развија, систем извршава и његови делови распоређују.

Предмет рада је поређење три архитектуралне варијанте система за обраду видео записа. Систем прихвата видео датотеку, издваја метаподатке, формира послове за израду рендиција¹ у задатим резолуцијама и обавља транскодовање² помоћу алата FFmpeg. Посматрани ток повезује јавни програмски интерфејс, трајно чување података, асинхрону обраду, спољни процес и складиште датотека. Он зато омогућава да се архитектуралне разлике посматрају дуж целог пословног тока, а не само кроз трајање једног синхроног захтева.

Опште препоруке, попут предлога да се нов систем под одређеним условима најпре развија као монолит [5], не предвиђају понашање конкретних имплементација. Мотив рада је зато да се три архитектуралне варијанте истог система упореде под заједничким условима и да се последице раздвајања модулских, процесних и распоредних граница повежу са непосредним доказима.

¹Рендиција је транскодовани излаз видео записа са задатим параметрима, као што је циљна резолуција.

²Транскодовање је промена начина кодовања или параметара медијског записа.

1.2 Циљ и истраживачка питања

Циљ рада је да постави контролисано поређење архитектуралне организације, мерљивог понашања и промена насталих преласком од монолита, преко модуларног монолита, до микросервиса, а затим да одговори на истраживачка питања само у мери коју допуштају доступни докази. Прва етапа уводи израженије модулске границе у заједничкој јединици распоређивања, док друга раздваја пословне одговорности у засебне сервисе.

Квантитативно поређење три конкретне имплементације обухвата четири експерименталне варијанте: монолитну варијанту, варијанту модуларног монолита и микросервисне топологије T1 и T2. Експериментална варијанта није изолована архитектурална ознака, већ обухвата архитектуралну организацију и топологију распоређивања под једном заједничком границом процесора и меморије. Код микросервисне имплементације додатно се пореде топологија са једном и топологија са две инстанце сервиса за транскодовање, уз непромењен укупан буџет и непромењен збир радних нити. Инстанца је један покренути примерак сервиса.

Истраживање је постављено око три питања:

- ИП1. Какве се разлике у пропусности, трајању оптерећења, латенцији успешне групе, употреби процесора и употреби меморије јављају између четири експерименталне варијанте при истом оптерећењу и у дефинисаним ресурсним профилима?³
- ИП2. Које се разлике у организацији кода, предусловима и топологији распоређивања, као и механизмима отказа и границама поузданости, могу утврдити из кода, конфигурације и доступних провера?
- ИП3. Које врсте документованих промена настају у обе етапе миграције, а какав је нормализовани описни обим друге етапе, исказан бројем јединица по категорији и категоријама захваћених артефаката?

Пропусност,⁴ трајање оптерећења и латенција успешне групе⁵ односе се на

³Ресурсни профил је унапред одређена комбинација укупног буџета процесора и меморије и дозвољене конкурентности апликационог слоја.

⁴Пропусност је број успешно завршених пословних операција у јединици времена. И операција и мерни интервал морају бити унапред одређени.

⁵Латенција успешне групе је време од прихватања захтева до успешног завршетка целе тражене групе, укључујући чекање. Латенција рендиције по резолуцији мери време до успешног завршетка појединачне рендиције и користи се као секундарна мера.

асинхрони пословни ток, а не само на трајање појединачног HTTP захтева. Њихове операционе дефиниције морају бити једнаке за све варијанте [7, 9].

1.3 Обим и приступ

Студија је постављена као контролисано поређење три архитектуралне варијанте. При промени архитектуре треба очувати домен, функционални обим, јавни програмски интерфејс у обиму четири операције главног тока, логику транскодовања, тест податке, сценарио оптерећења и окружење мерења. Намерно се разликују модулске и процесне границе, комуникација и топологија распоређивања. Методологија мерења захтева да систем под тестом, фактор, контролне променљиве, оптерећење и метрике буду унапред одређени [7, 9].

Квантитативни део обухвата независно поновљена мерења под једнаким укупним ресурсним буџетом који једна заједничка ресурсна група намеће целом распоређивању, без ужих граница процесора и меморије за њене чланове. Квалитативни део примењује заједничке критеријуме на организацију кода, топологију распоређивања, проверљиве механизме отказа и границе поузданости. Анализа миграције користи документоване семантичке промене. Без посебних временских записа из њих се не изводе тврдње о уложеном раду, трајању или трошку.

Обим је ограничен на транскодовање видео записа, изабрани технолошки скуп, синтетичке тест податке, дефинисано оптерећење и локално окружење. Израда слика, издвајање звука, продукционо оптерећење и окружење са оркестрацијом нису предмет експеримента. Ова ограничења одређују и домет завршних препорука.

1.4 Намеравани допринос и структура рада

Допринос рада чине упоредив скуп квантитативних података за три архитектуралне варијанте и две микросервисне топологије, операционе дефиниције пропусности, латенције успешне групе и латенције рендиције по резолуцији и документована двоетапна миграција. Посебан део доприноса је поређење унутар једног заједничког ресурсног буџета, уз раздвајање вертикалног повећања ресурса од промене броја инстанци сервиса за

транскодовање. Ова синтеза омогућава условне закључке о конкретним имплементацијама система са сродним захтевима и условима.

После увода, Поглавље 2 излаже теоријске основе и сродне радове. Методологија је у Поглављу 3. Варијанте и две етапе миграције описане су у Поглављима 4, 5 и 6. Резултати и дискусија су у Поглављу 7, а закључак у Поглављу 8.

Глава 2

Теоријске основе

2.1 Критеријуми класификације

Софтверска архитектура обухвата структуре система, односе њихових елемената и својства значајна за разумевање и развој [3]. Зато се не одређује само на основу програмског оквира, инфраструктурног производа или броја компонената. У овом раду варијанте се класификују према модулским, процесним и распоредним границама, начину комуникације, власништву над подацима и операционим последицама.

Посебно је важно разликовати три појма. *Модулска граница* раздваја одговорности и дозвољене зависности унутар кода. *Граница процеса* раздваја извршавање у различитим адресним просторима. *Јединица распоређивања* је артефакт или део система који се објављује и поставља у рад као целина.⁶ Ове границе могу да се подударају, али нису међусобно заменљиве.

2.2 Монолит

Монолит обједињује пословне компоненте у заједничкој јединици распоређивања, па измена једне компоненте типично захтева поновно распоређивање целине [2, 6]. Компоненте нису независно распоредиве, али систем може имати јасну унутрашњу структуру и више модула. Зато одсуство модула није услов ове класификације.

⁶Део је независно распоредив ако се може изменити и поставити у рад без поновног постављања осталих делова. Јединица распоређивања није нужно исто што и артефакт градње, процес или контејнер.

Број процеса није универзална дефиниција монолита. За класификацију је пресудна заједничка јединица распоређивања и одсуство независно распоредивих пословних компонената. Конкретна монолитна варијанта овог рада извршава се као један апликациони процес по инстанци. То је својство имплементације које се проверава у Поглављу 4.

2.3 Модуларни монолит

Модуларни монолит задржава заједничку јединицу распоређивања, али код организује у слабо спрегнуте модуле са јасним одговорностима, јавним интерфејсима и експлицитним зависностима [8]. За његово разликовање од полазног монолита потребни су докази о модулима, објављеним интерфејсима, дозвољеним зависностима и очуваном заједничком распоређивању.

Аутоматска провера зависности није сама по себи дефиниција обрасца, већ својство конкретне имплементације. Алат Spring Modulith може из структуре пакета извести модел модула, описати именоване интерфејсе и проверити циклусе, приступ интерним пакетима и задата правила зависности [4, 10]. Присуство алата није довољан доказ, јер су потребни стварна правила и поновљив излаз њихове провере.

2.4 Микросервисна архитектура

Микросервисна архитектура организује систем као скуп кохезивних сервиса који се извршавају у засебним процесима, комуницирају порукама и могу се независно распоређивати [6]. Величина сервиса није довољан критеријум. Одлучујуће су независне распоредне границе и уговори сарадње [2].

Комуникација преко границе процеса⁷ уводи могућност недоступности учесника или везе и додатни трошак преноса [2, 6]. Зато су уговори, поступање при отказу и могућност увида у рад система део поређења. Децентрализовано управљање подацима јесте честа особина, али не захтева нужно засебну физичку базу за сваки сервис [2]. У овом раду власништво

⁷Комуникација преко границе процеса одвија се посредством мрежног или другог међупроцесног механизма, а не непосредним позивом функције у истом адресном простору.

над подацима значи да је један сервис одговоран за пословно значење и измену одређеног скупа података. Оно се проверава кроз дозвољене приступе и трансакционе границе, а не само кроз број физичких база.

2.5 Синтеза критеријума

Табела 2.1 сажима критеријуме који се касније примењују на проверљиве артефакте сваке варијанте.

Критеријум	Монолит	Модуларни монолит	Микросервиси
Јединица распоређивања	Заједничка целина. Пословне компоненте нису независно распоредиве [2, 6].	Заједничка целина за све модуле [8].	Сервиси су независно распоредиви [2].
Унутрашње границе	Нису дефинициони услов. Модули могу постојати.	Јавни интерфејси и експлицитне зависности модула [8].	Уговори раздвајају сервисе [2].
Процесна граница	Пословне компоненте посматране варијанте извршавају се у заједничком апликационом процесу.	Модули деле апликациони процес [8].	Сервиси имају засебне процесе [6].

Табела 2.1: Критеријуми разграничења архитектуралних варијанти

За потребе миграције, *граница по конвенцији* означава поделу изражену пакетима, именовањем или документацијом. *Аутоматски проверљива граница* додатно има формализовано правило и поновљив поступак који открива његово нарушавање. У овој студији тај поступак се ослања на проверу зависности коју подржава алат Spring Modulith [10]. Каснија анализа зато раздваја постојање поступка од архивираног исхода његовог извршавања.

Архитектурални образац описује организацију и ограничења система, док алати и инфраструктура омогућавају њихову реализацију или проверу [3].

Зато ни појединачан алат, број контејнера или избор посредника за поруке сами не одређују архитектуру.

Класификација обухвата стварне модулске, процесне и распоредне границе и уговоре. На тој основи ИП1 пореди мерљиво понашање варијанти. ИП2 разматра њихове организационе и операционе последице. ИП3 прати промене између узастопних архитектуралних облика.

Глава 3

Методологија

Методологија операционализује три истраживачка питања. Поређење је контролисано. Систем под тестом, експерименталне варијанте, оптерећење, метрике и прикупљање података одређују се пре тумачења исхода. Експериментална варијанта обухвата архитектуралну организацију и топологију распоређивања под заједничком ресурсном границом. Архитектура сама није изолован фактор. Тако се промена експерименталне варијанте не меша са променом улаза или мерног окружења [7, 9].

3.1 Дизајн студије и систем под тестом

Систем под тестом⁸ обухвата цело распоређивање потребно за пријем и транскодовање видео записа. Пореде се четири експерименталне варијанте: монолитна варијанта, варијанта модуларног монолита, микросервисна топологија T1 са једном инстанцом сервиса за транскодовање и микросервисна топологија T2 са две инстанце.

Заједнички функционални ток обухвата отпремање видео записа пре мерења, захтев за израду рендиција и периодично читање статуса до коначног исхода. Скуп операција главног јавног тока, пословна значења исхода и ваљаност излазних видео записа проверавају се истим функционалним поступком за све варијанте.

Једнакост главног јавног тока одређују четири операције. Отпремање је вишеделни захтев POST на путањи `/api/v1/video`, са пољем `file`, одговором

⁸Систем под тестом је скуп компонената чије се понашање пореди под контролисаним условима.

201 и подацима видео записа. Захтев за израду је POST на `/api/v1/rendition`, са пољима `videoId` и `resolutions` и успешним статусом 200. Стање се чита операцијом GET на `/api/v1/rendition/video/{videoId}`, која са статусом 200 враћа списак рендиција са њиховим идентификатором, резолуцијом, стањем и временским ознакама. Завршена рендиција преузима се операцијом GET на `/api/v1/rendition/{renditionId}/download` као бинарно тело са статусом 200. Ресурсни профил наметнут је једној родитељској ресурсној групи у јединственој хијерархији `sgroup v2`⁹. Код монолита и модуларног монолита она обухвата апликацију и PostgreSQL. Код микросервиса обухвата мрежни пролаз, сервис за медијске садржаје, сваку инстанцу сервиса за транскодовање, сервис за обавештења, RabbitMQ и PostgreSQL. Алат за стварање оптерећења и мерни алати налазе се изван ове границе. Конфигурација званичног протокола није задавала уже границе процесора или меморије појединачним члановима родитељске ресурсне групе, док је родитељска група наметала укупан буџет процесора и меморије.

⁹То је друга верзија Линуксовог механизма за хијерархијско груписање процеса и наметање заједничких ресурсних граница.

3.2 Експериментално окружење

Експерименти су изведени на једном рачунару у локалном окружењу. Његова хардверска и софтверска основа приказана је у Табели 3.1.

Ставка	Вредност
Модел процесора	Intel Core Ultra 7 155H
Процесорска конфигурација	16 језгара, 22 логичка процесора
Радна меморија	16 гибибајта
Оперативни систем	Fedora Linux 43
Језгро оперативног система	Linux 7.1.4
Контејнерска платформа	Docker 29.6.2
Састављање контејнерског окружења	Docker Compose 5.3.1
Извршно окружење	Temurin Java 21.0.11
Апликациони програмски оквир	Spring Boot 4.0.6
Подршка за модуларност	Spring Modulith 2.0.7
Алат за стварање оптерећења	Gatling 3.11.5
Систем за управљање базом података	PostgreSQL 17
Посредник порука	RabbitMQ 4.1
Мрежни пролаз	Nginx 1.27
Прикупљање метрика	Prometheus 2.53.4
Метрике контејнера	cAdvisor 0.49.1
Приказ метрика	Grafana 11.6.0
Алат за транскодовање	FFmpeg 8.0.1

Табела 3.1: Хардверска и софтверска основа експерименталног окружења

3.3 Променљиве, оптерећење и ресурсни профили

Контролне променљиве су функционални ток и скуп његових јавних операција, садржај и трајање улаза, тражени излази, конфигурација транскодовања, рачунар и мерно окружење. Избором експерименталне варијанте намерно се мењају модулске и процесне границе, механизми комуникације и топологија распоређивања.

Профил `scaling-4c-4g` заједничкој групи додељује 4 процесорска језгра и 4 гибибајта меморије, а профил `scaling-8c-8g` додељује 8 језгара и 8 гибибајта. Агрегатна конкурентност транскодовања је редом четири и осам радних нити. Монолит, модуларни монолит и микросервисна топологија T1 користе један скуп радних нити. У T2 су нити равномерно подељене између две инстанце сервиса за транскодовање: по две у мањем, односно по четири у већем профилу. Поређење T1 и T2 зато мења топологију, док процесор, меморија и агрегатна конкурентност остају фиксни.

Свако покретање користи 12 међусобно независних, детерминистичких видео записа трајања 10 секунди, резолуције 1920×1080 и кодовања H.264. Они се отпремају пре мерног интервала. За сваки видео траже се SD_360, SD_480 и HD_720, што даје 36 исцрпно праћених пословних исхода. Секвенцијално оптерећење завршава све три рендиције једног видео записа пре подношења следећег захтева, док паралелно оптерећење подноси захтеве за свих 12 видео записа без тог чекања.

Оба начина су изабрана да осветле различите услове подношења истог рада. Секвенцијално подношење показује обраду једне групе уз минимално надметање у реду, док паралелно, нагло подношење оптерећује формирање реда, распоређивање и надметање унутар заједничког скупа ресурса.

Варијанте се не покрећу паралелно. Утицај променљивог стања рачунара и редоследа извршавања умањује се тридесетосекундном стабилизацијом у мировању, независним понављањима и уравнотеженим редоследом [7, 9, 1].

3.4 Операционе дефиниције метрика

Метрика мора да одреди операцију, јединицу, мерни интервал и правило укључивања [7, 9]. Табела 3.2 даје дефиниције које се једнако примењују на све варијанте.

У овом раду радни скуп означава вредност метрике `container_memory_working_set_bytes` коју излаже алат cAdvisor. За сваки временски тренутак сабирају се вредности свих чланова ресурсне границе.

Мера	Операциона дефиниција	Јединица
Трајање оптерећења	Време од најранијег почетка мерења било које јединице рада до завршетка последње од свих 12 група, односно свих 36 рендиција. Мера је доступна само када су све рендиције успешно завршене.	секунде
Пропусност	Свих 36 успешно завршених рендиција подељено трајањем оптерећења. Не израчунава се из делимичног исхода.	завршене рендиције у секунди
Латенција успешне групе	Време од прихватања захтева до завршетка све три планиране рендиције, само за потпуно успешне групе.	секунде
Латенција рендиције по резолуцији	Време од прихватања захтева до исхода FINISHED , засебно по резолуцији. Ово је секундарна дијагностичка мера.	секунде
Употреба процесора	Средња вредност агрегата употребе процесора свих чланова ресурсне границе.	процесорска језгра
Употреба меморије	Средња вредност агрегата радних скупова свих чланова ресурсне границе. Највећа вредност задржава се као техничка дијагностика.	бајтови

Табела 3.2: Операционе дефиниције квантитативних метрика

Намераваани излаз је једна тражена рендиција видео записа. За примарне метрике успешне обраде улазе само исходи **FINISHED**. Латенција успешне групе додатно захтева да су завршене све три планиране рендиције истог видео записа. Број захтева у секунди није пословна пропусност. Подаци алата Gatling о преносу обухватају и припремно отпремање датотека, па се чувају као дијагностика целе симулације, али се искључују из званичних групних статистика и графикана. Употреба процесора и меморије изводи се из временски поравнатих серија контејнера који припадају ресурсној граници. За сваки тренутак сабирају се вредности свих апликационих и инфраструктурних чланова, а средња вредност тих збирова представља једно покретање. То је агрегат чланова ресурсне границе, а не непосредно читавање бројача родитељске контролне групе. Узорци унутар покретања нису независна понављања.

Латенција успешне групе је главна мера латенције у упоредном приказу и изражава се у секундама. Латенција рендиције по резолуцији, такође изражена у секундама, остаје секундарни дијагностички доказ за проверу поретка $SD_{360} < SD_{480} < HD_{720}$, без захтева да се све њихове бројчане вредности прикажу у поглављу са резултатима. За меморију се упоредно приказује средњи радни скуп, док се највећа вредност чува у техничким доказима ради дијагностике.

3.5 Извођење експеримента и обрада резултата

Свако покретање спаја једну експерименталну варијанту, ресурсни профил, начин оптерећења и редни број понављања. Четири експерименталне варијанте испитују се у два профила и при два начина оптерећења, са по пет понављања. Званична матрица зато обухвата $4 \times 2 \times 2 \times 5 = 80$ независних покретања у 16 експерименталних комбинација.

Пре сваког покретања припрема се стање система и отпрема 12 датотека, након чега следи 30 секунди стабилизације у мировању. За свако покретање проверавају се доступност компонената пре мерења и целовитост прикупљених података по његовом завршетку. Редослед експерименталних варијанти и профила уравнотежен је да њихов положај не би био увек исти. Пробна покретања искључена су из анализе, а протокол, параметри, распоред и поступак анализе утврђени су пре извођења званичне матрице.

Мерни интервал почиње са обрадом прве јединице рада и траје до завршетка последње од 12 видео група, односно последње од 36 рендиција. За свако покретање бележе се временске ознаке, исходи и временске серије употребе ресурса.

Метрике се најпре сажимају на нивоу покретања. Свака комбинација експерименталне варијанте, профила и начина оптерећења затим се описује бројем покретања за која је дата метрика доступна, аритметичком средином, медијаном, узорачком стандардном девијацијом, минимумом и максимумом.

Графикони приказују појединачна понављања, средњу вредност и 95% интервал поверења заснован на Студентовој t расподели.¹⁰ Интервал показује

¹⁰Ова расподела се користи при процени неизвесности средње вредности на основу малог узорка када варијанса популације није позната.

неизвесност процене средње вредности, док појединачне тачке и њихова распршеност показују променљивост између покретања [7, 9, 1]. Анализа је описна: не спроводе се тестови значајности или еквиваленције, профили се не спајају у јединствен ранг, нити се за неизмерене услове интерполирају или процењују вредности.

Пошто се у сваком покретању обрађује исти скуп од 36 рендиција, пропусност и трајање оптерећења представљају два приказа истог фиксног обима рада, а не независну потврду истог налаза.

Планирана поређења полазе од средњих вредности два профила унутар исте експерименталне варијанте и начина оптерећења. У релативном приказу пропусности монолит има референтну вредност од 100% унутар истог профила и начина оптерећења. Микросервисне топологије T2 и T1 пореде се при истом укупном ресурсном буџету и једнаком агрегатном броју радних нити.

3.6 Квалитативно поређење и евиденција миграције

За ИП2 се на све варијанте примењују заједнички критеријуми:

- организација кода описује се прегледом одговорности, јавних интерфејса, зависности и машински проверљивих правила,
- распоређивање се описује бројем и улогом пословних јединица, инфраструктурним предусловима и спроведеним границама података,
- механизми отказа утврђују се из трансакционих и комуникационих граница, без приписивања опоравка који није извршно проверен.

Обим ИП2 ограничен је на механизме које непосредно показују код, конфигурација и доступне извршне провере појединачних токова. Симетричан експеримент опоравка није изведен, па се из статичког прегледа не изводе квантитативни закључци о времену или степену опоравка.

За ИП3 је јединица анализе *документована миграциона промена*: једна семантичка промена са циљем, полазном и циљном варијантом, категоријом и верзијама имплементације или путањама до пратећих артефаката. Више датотека које остварују исти циљ групишу се у једну промену, а премештање

без промене значења бележи се засебно. За другу етапу је по овом правилу вођен нормализовани дневник. Прва етапа нема дневник нормализован по истом правилу, већ је реконструисана ретроспективно и описана квалитативно, па се кардиналности двеју етапа не пореде. Број датотека и линија може бити описни атрибут, али није мера труда. Нормализовани описни обим одређује се само за другу етапу, као број документованих јединица по категорији и скуп категорија захваћених артефаката. Он не представља време, трошак или уложени рад.

3.7 Синтеза, валидност и поновљивост

Препоруке се изводе повезивањем налаза од ИП1 до ИП3 са практичним приоритетом, испитаним профилем, ограничењем и условом примене. Не формира се јединствена оцена без образложених тежина. Препорука је оправдана само ако је њен предуслов непосредно мерен или посматран.

Конструктну валидност угрожава замена пословне пропусности бројем захтева или временске употребе ресурса тренутним снимком. Зато су мере операционо раздвојене. Унутрашњу валидност угрожавају редослед, променљиво стање рачунара и истовремена промена више својстава ресурсног профила. Уравнотежен редослед, чисто стање, стабилизација у мировању, независна понављања, једна збирна ресурсна граница и тумачење профила као целине умањују ове утицаје [1]. Ипак, архитектурална организација и топологија заједно образују сложен експериментални фактор, па се уочене разлике не приписују једном изолованом узроку.

Протокол није садржао загревање радним сценаријом. Зато резултати могу да обухвате учитавање класа, динамичко превођење и друге ефекте првог оптерећења, који се могу разликовати између једне и више виртуелних машина Java. Мерења описују цео задати сценарио, али не успостављају перформансе стабилног стања после загревања.

Спољашња валидност ограничена је доменом транскодовања, изабраним технолошким скупом, синтетичким улазом, дефинисаним оптерећењем и локалним окружењем. Налази се односе на конкретне имплементације под испитаним условима. Закључну валидност ограничавају пет независних понављања по комбинацији, варијабилност међу покретањима и више исхода и поређења. Зато се приказују сва понављања и описна неизвесност.

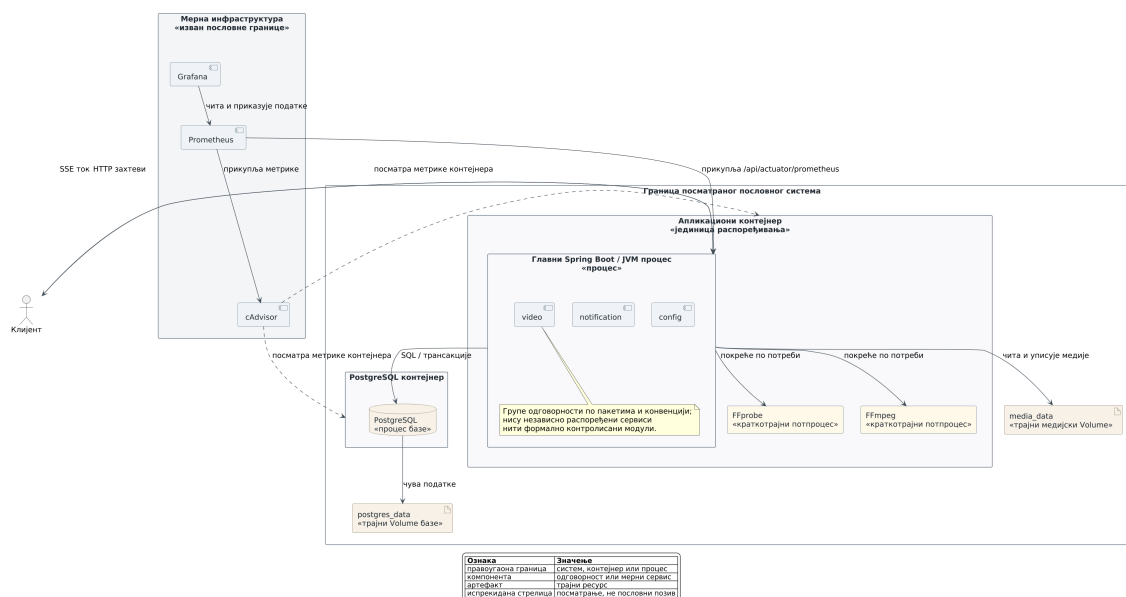
Овако постављен метод раздваја последице модулских, процесних и распоредних граница од ограничења доступних доказа и не претвара описно поређење конкретних имплементација и топологија у узрочну тврдњу о самој архитектури.

Глава 4

Монолитна архитектура

Монолитна варијанта је полазна имплементација у миграцији. Све пословне одговорности граде се у једну извршиву архиву апликације Spring Boot и распоређују као целина. Један апликациони процес по потреби покреће краткотрајне потпроцесе FFprobe и FFmpeg, док се база PostgreSQL извршава у засебном инфраструктурном контејнеру. Монолитна класификација стога почива на једној пословној јединици распоређивања [2, 6].

Именовани простор за складиштење медијских датотека, база и мерна инфраструктура налазе се изван пословне границе. Њихов однос према апликационом процесу приказан је на Слици 4.1.



Слика 4.1: Граница система и инфраструктурне зависности монолитне варијанте

4.1 Организација и јавни програмски интерфејс

Код је груписан према одговорностима за видео записе, обавештења и конфигурацију. Приступ складишту, обради, објављивању догађаја и чувању података изражен је портовима и адаптерима.¹¹ Таква организација раздваја апликационе интерфејсе од инфраструктурних реализација унутар исте јединице распоређивања.

Јавне операције, настале спајањем контекстне путање `/api` са мапирањима контролера, приказане су у Табели 4.1.

Глобални руковалац пресликава пословне изузетке у одговоре са статусима 400, 404, 409, 422 и 500. Стање рендиције описују вредности `PENDING`, `RUNNING`, `FINISHED`, `CANCELED` и `ERROR`, док главни ток обраде користи све наведене вредности осим `CANCELED`.

¹¹Порт је интерфејс преко ког језгро апликације тражи или пружа услугу, док адаптер тај интерфејс повезује са конкретном технологијом.

Метод	Путања	Намена
POST	/api/v1/video	Отпремање видео датотеке као вишеделног захтева.
GET	/api/v1/video	Списак видео записа.
GET	/api/v1/video/{videoId}	Видео запис са припадајућим рендицијама.
POST	/api/v1/rendition	Захтев за израду једне или више рендиција.
GET	/api/v1/rendition/video/{videoId}	Стања рендиција једног видео записа.
GET	/api/v1/rendition/{renditionId}/download	Преузимање завршене рендиције.
GET	/api/v1/notification	Списак забележених обавештења.
GET	/api/v1/notification/stream	Ток серверских догађаја са обавештењима.

Табела 4.1: Јавне операције монолитне варијанте

4.2 Пријем и чување видео записа

При отпремању сервис привремено чува датотеку, алатом FFprobe издваја метаподатке, премешта је у трајни директоријум и у бази чува видео запис са путањом до датотеке.

Датотечни систем и база чине две границе трајности, при чему једна чува медијски садржај, а друга метаподатке и путању. Исто раздвајање важи за излазну датотеку рендиције и њен запис у бази.

4.3 Асинхрона израда рендиције

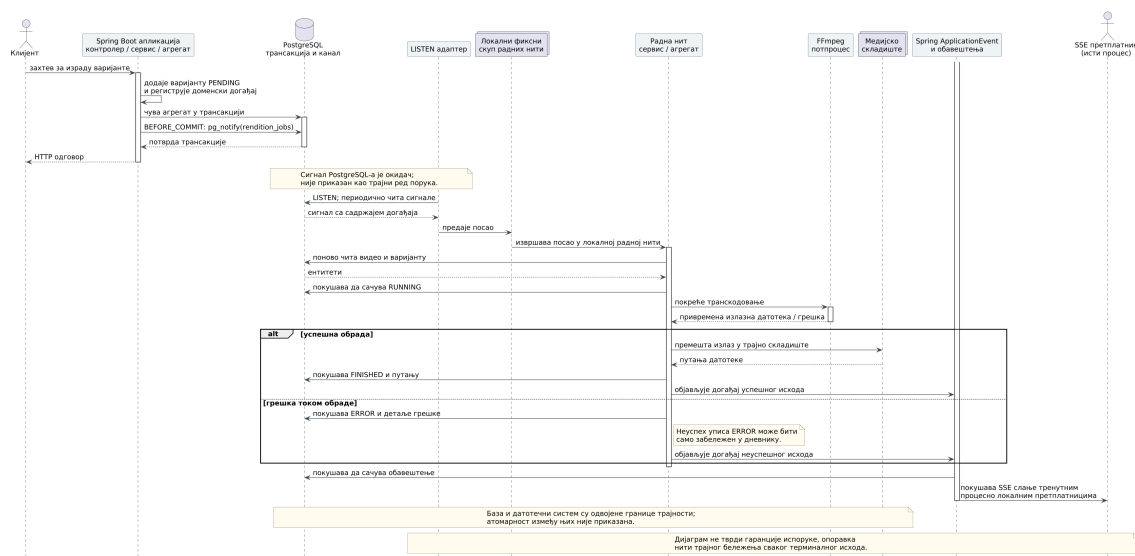
Захтев за израду додаје рендицију у стању PENDING и региструје доменски догађај. При чувању трансакциони слушалац пре потврде трансакције позива `pg_notify` над каналом `rendition_jobs`. Механизам LISTEN/NOTIFY¹² повезује упис посла са локалним покретањем. Веза истог процеса слуша канал и предаје посао скупу радних нити фиксне величине. Трајни опис посла остаје запис рендиције у бази.

Радна нит чита видео запис и рендицију, поставља стање RUNNING и покреће FFmpeg. Излаз се премешта у трајни директоријум, а његова путања

¹²У овој имплементацији називи означавају регистрацију везе за канал и слање сигнала са садржајем догађаја.

чува уз стање **FINISHED**. При грешци руковалац покушава да упише стање **ERROR** и поруку грешке.

После исхода обраде руковалац механизмом `Spring ApplicationEvent` објављује догађај унутар процеса. Слушалац чува обавештење, а затим га шаље тренутно пријављеним претплатницима тока серверских догађаја.¹³ Цео ток, од покретања посла до обавештавања, остаје у једном апликационом процесу и приказан је на Слици 4.2.



Слика 4.2: Секвенца израде рендиције и обавештавања у монолитној имплементацији

4.4 Конкурентност и границе комуникације

Број радних нити задаје се својством `rendition.pool-size`, чија је подразумевана вредност осам, док се у мерним покретањима она замењује бројем одређеним ресурсним профилем. Руковалац у меморији одржава скуп активних парова видео записа и резолуција и спречава поновно покретање истог активног посла у тој инстанци. И заштита од дуплираног покретања и скуп претплатника припадају једном процесу.

¹³Ток серверских догађаја у овој имплементацији је дуготрајан HTTP одговор којим сервер шаље именована обавештења пријављеном клијенту.

4.5 Распоређивање и инструментација

Контејнерска слика садржи извршиву архиву и алате за обраду. Опис распоређивања повезује апликацију са базом и засебним просторима за податке базе и медијске датотеке. Ограничења процесора и меморије, као и број радних нити, задају се променљивама окружења.

Апликација излаже приступне тачке за проверу здравља и метрике. Поред метрика захтева, региструје број рендиција у стању PENDING, број активних послова и трајање путање израде, које обухвата и чекање и обраду. Засебна мерна конфигурација повезује апликацију са алатима Prometheus, cAdvisor и Grafana.

У монолитној варијанти одговорности, локална комуникација и управљање рендицијама обједињени су у једном процесу и једној пословној јединици распоређивања. Због заједничке процесне и распоредне границе, унутрашње раздвајање не омогућава независно распоређивање пословних одговорности.

Глава 5

Модуларни монолит

Ова варијанта задржава један апликациони процес и једну јединицу распоређивања, али пословни код дели на модуле са различитим одговорностима и објављеним интерфејсом. Та комбинација одговара класификацији модуларног монолита [8]. У односу на полазну варијанту, прва миграциона етапа помера границе у организацији кода, док процесна и распоредна граница остају заједничке, а оба пословна модула користе исту физичку базу.

5.1 Модулске одговорности и објављени интерфејс

Извршни пакети `video` и `notification` представљају пословне модуле, док `config` и `shared` обезбеђују техничку конфигурацију и заједничку основу. Њихове одговорности приказане су у Табели 5.1.

Пакет `video.published` је именовани интерфејс `published`.¹⁴ Он садржи три типа догађаја о исходу обраде. Модул `notification` зависи од овог интерфејса, а не од интерних типова модула `video`. Тиме је међумодулска сарадња изражена на начин који подржава модел алата Spring Modulith [10].

¹⁴Именовани интерфејс је јавно означен део модула који други модул може да користи.

Модул	Уочена одговорност	Међумодулски интерфејс
<code>video</code>	Пријем и чување видео записа, управљање изработом рендиција и покретање обраде.	Објављени интерфејс <code>video::published</code> са три типа догађаја о исходу обраде.
<code>notification</code>	Чување, читање и слање обавештења активним претплатницима.	Користи типове догађаја из интерфејса <code>video::published</code> .
<code>config</code>	Везивање путање складишта и укључивање периодичног извршавања.	Техничка конфигурација коју користе адаптери модула <code>video</code> .
<code>shared</code>	Основне класе ентитета са идентификаторима и временским ознакама.	Заједничка техничка основа модела модула <code>video</code> и <code>notification</code> .

Табела 5.1: Одговорности модула у варијанти модуларног монолита

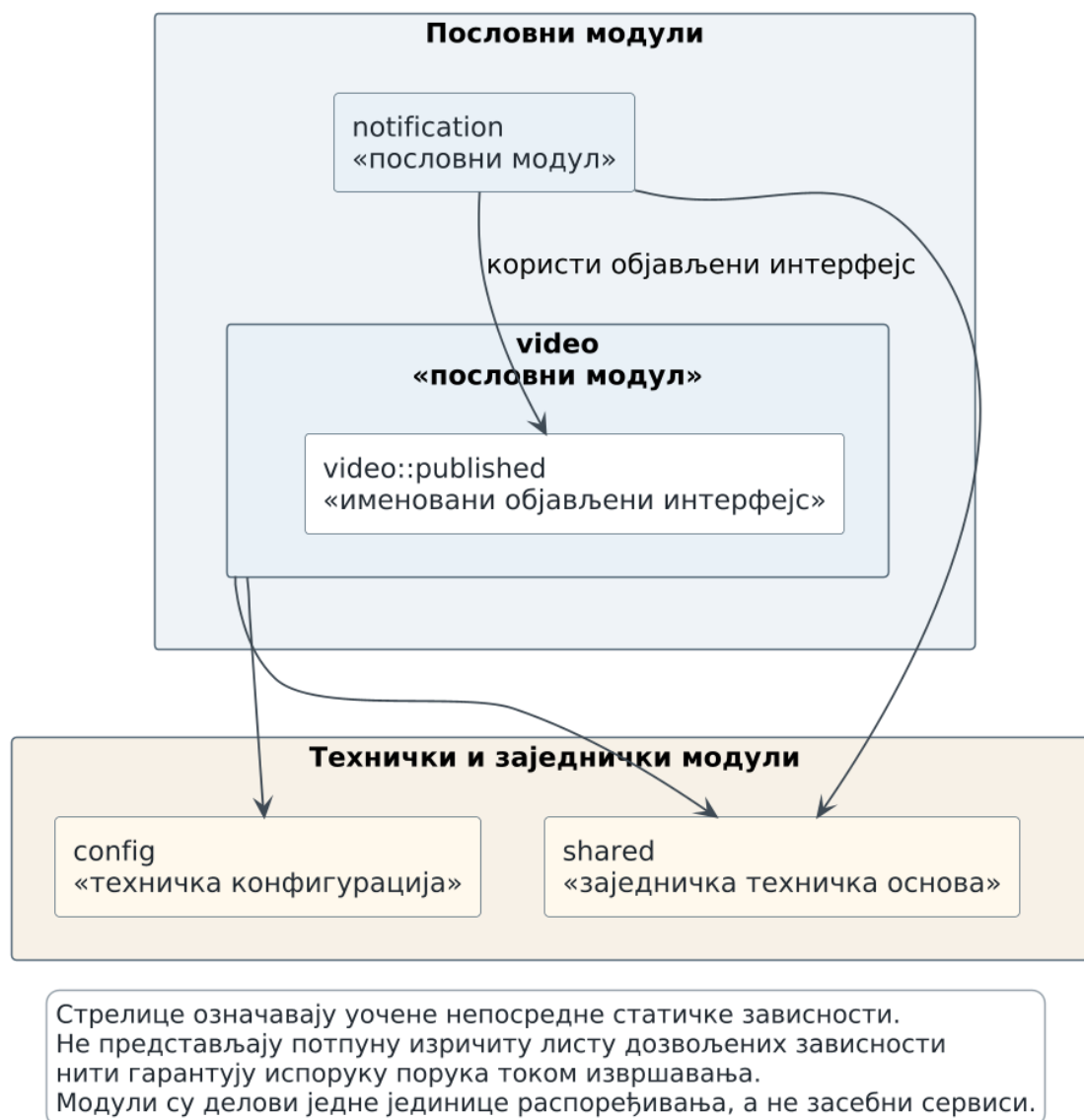
5.2 Зависности и провера модулских граница

Табела 5.2 приказује четири непосредне зависности утврђене из увоза типова и наслеђивања у главном изворном коду.

Уочена ивица	Непосредан траг у коду
<code>notification</code> → <code>video::published</code>	Слушалац обавештења увози три објављена типа догађаја модула <code>video</code> .
<code>notification</code> → <code>shared</code>	Ентитет обавештења наслеђује заједничку основну класу агрегата.
<code>video</code> → <code>shared</code>	Ентитети видео записа и рендиција наслеђују класе из модула <code>shared</code> .
<code>video</code> → <code>config</code>	Адаптери складишта и транскодовања користе подешавање путање складишта.

Табела 5.2: Уочене непосредне зависности између модула

Слика 5.1 сажима ове везе. Тест `ModularityTests` гради модел модула и позивом `modules.verify()` проверава циклусе и приступ нејавним деловима модула [10]. Тест модулских граница извршен је у окружењу Java 21 и прошао је без неуспеха.



Слика 5.1: Уочене статичке зависности пословних и техничких модула у варијанти модуларног монолита

5.3 Јавни програмски интерфејс и ток обраде

Модуларни монолит задржава скуп операција главног јавног тока дефинисан у Поглављу 3, који обухвата отпремање видео записа, захтев за израду рендиција, периодично читање стања и преузимање резултата. Исти сценарио зато може да се примени без промене клијентских операција.

Декларисани скуп стања садржи PENDING, RUNNING, FINISHED, CANCELED

и `ERROR`, а главни ток обраде остаје као у Поглављу 4. Архитектурална промена настаје на граници преко које се исход обраде објављује модулу за обавештења.

5.4 Комуникација и границе података

Руководилац послом у модулу `video` ствара тип догађаја из интерфејса `video::published`. Адаптер га предаје механизму `Spring ApplicationEvent Publisher`, а модул `notification` га прима методама означеним са `@EventListener`. Сви учесници припадају истом апликационом процесу, па је међумодулска комуникација локална.

Сервис за обавештења обрађује трајни упис обавештења пре слања активним претплатницима.

Покретање посла унутар модула `video` и даље користи `pg_notify` и канал базе `PostgreSQL`. Тај ток покреће обраду, док објављени модулски интерфејс преноси њен исход модулу за обавештења.

Ентитети модула `video` и `notification` користе исти извор података и исту физичку базу. Обавештење чува идентификаторе видео записа и рендиције без објектне релације ка ентитетима модула `video`. Модулска граница је изражена у коду, а подаци оба модула остају у истој физичкој бази.

5.5 Распоређивање и инструментација

Модулска подела задржава једну апликациону целину, базу `PostgreSQL` и трајне просторе за податке базе и медијске датотеке. Мерну инфраструктуру чине `Prometheus`, `cAdvisor` и `Grafana`. Апликација излаже приступне тачке за проверу здравља и метрике, а ознаке `variant=modular` и `service=media-modular` разликују њене временске серије.

5.6 Прва етапа миграције

Ретроспективни опис прве етапе издваја три врсте промене, и то издвајање заједничких основних класа, увођење објављеног интерфејса за догађаје и додавање поступка за проверу и приказ модулске структуре. Ове врсте

нису нормализоване миграционе јединице и не користе се за квантитативно поређење са другом етапом.

Прва миграција изражава границе унутар кода, али задржава локалну комуникацију, једну базу, један процес и једну јединицу распоређивања. Њена архитектурална последица је строжа контрола међумодулских зависности без увођења расподељене комуникације и независног распоређивања.

Глава 6

Микросервисна архитектура

Микросервисна варијанта раздваја заједничку процесну и распоредну границу на три пословна сервиса у односу на модуларни монолит из Поглавља 5. Сваки сервис има засебан извршни артефакт, контејнерску слику, процес и одговорност, док се сарадња при изради рендиције одвија порукама. Ова својства одговарају критеријумима микросервисне архитектуре [2, 6]. Друга миграциона етапа тако помера модулске границе на ниво процеса и распоређивања, а локалну комуникацију замењује разменом порука.

Јавне операције под путањама `/api/v1/video`, `/api/v1/rendition` и `/api/v1/notification`, као и употреба алата FFprobe и FFmpeg, остају заступљени у имплементацији. Извршна провера једнакости ограничена је на четири операције главног тока из Поглавља 3. Мењају се процесне, комуникационе, трансакционе границе и границе података.

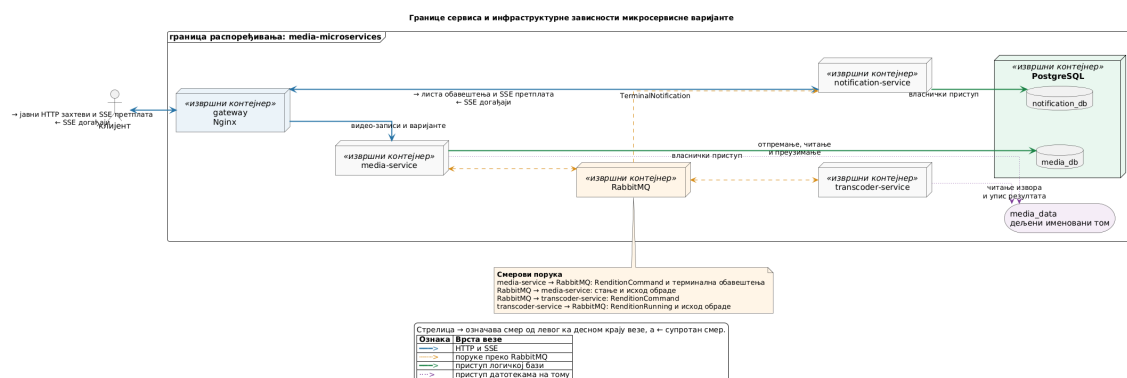
6.1 Топологија и јединице распоређивања

Јавни саобраћај улази кроз мрежни пролаз Nginx, који операције над видео записима и рендицијама усмерава ка сервису `media-service`, а операције над обавештењима ка сервису `notification-service`. Трећи пословни сервис, `transcoder-service`, прима налоге за обраду и извршава FFmpeg. Посредник RabbitMQ преноси налоге, стања и терминална обавештења између сервиса. Један контејнер базе покреће једну инстанцу PostgreSQL, у којој се налазе логичке базе `media_db` и `notification_db`. Сервиси `media-service` и `transcoder-service` приступају дељеном простору

за складиштење `media_data`.

Заједнички модул `messaging-contract` садржи типове порука које користе три сервиса. Основна извршна топологија садржи мрежни пролаз Nginx, три пословна сервиса, посредник RabbitMQ и инстанцу PostgreSQL. У микросервисној топологији T2 покрећу се две инстанце сервиса за транскодовање, док остале улоге остају непромењене. Дељени простор `media_data` је трајни ресурс између два сервиса.

Слика 6.1 приказује распоредне границе, ток јавног саобраћаја, поруке и приступ заједничким датотекама.



Слика 6.1: Границе сервиса и инфраструктурне зависности микросервисне варијанте

6.2 Власништво над операцијама и подацима

Табела 6.1 сажима одговорности три пословна сервиса. Граница је изражена јавним операцијама, порукама и непосредно доступним подацима, а не само именом процеса.

Сервис	Одговорност и границе
media-service	Одговорност: Пријем видео записа, FFprobe, управљање рендицијама, стањима и преузимањем. Јавне операције и поруке: Јавне операције видео записа и рендиција. Сервис шаље налоге и обавештења и прима стања обраде. База и датотеке: media_db. Чита и пише media_data. Кључна граница: Власник медијских података и јавног стања рендиције.
transcoder-service	Одговорност: Извршавање FFmpeg обраде и упис излазне датотеке. Јавне операције и поруке: Прима налоге и шаље стања и исходе. База и датотеке: Пословне податке прима у поруци. Чита извор и пише излаз у media_data. Кључна граница: Власник извршавања транскодовања.
notification-service	Одговорност: Трајни упис, листање и слање обавештења активним претплатницима. Јавне операције и поруке: Јавне операције листе и тока серверских догађаја. Сервис прима терминална обавештења. База и датотеке: notification_db. Кључна граница: Власник података и јавног тока обавештења.

Табела 6.1: Власништво над операцијама и подацима у микросервисној варијанти

Две логичке базе имају засебне власничке налоге и права повезивања, а смештене су у истој инстанци PostgreSQL. Граница података тиме прати власништво сервиса на логичком, али не и на физичком нивоу.

Дељени простор за складиштење media_data омогућава да сервис за транскодовање чита извор и упише резултат без промене датотечног механизма коришћеног у претходним варијантама. Базе су логички раздвојене, док медијске датотеке остају дељена инфраструктурна граница сервиса за медије и транскодовање.

6.3 Јавни програмски интерфејс

Микросервисна варијанта излаже скуп операција главног јавног тока дефинисан у Поглављу 3 преко мрежног пролаза. Мрежни пролаз усмерава операције ка сервисима који су власници одговарајућих пословних одговорности, док клијентски сценарио остаје непромењен.

6.4 Ток порука и трансакционе границе

Када клијент затражи израду, `media-service` у локалној трансакцији уписује тражене рендиције у стању `PENDING`. Тек после потврде те трансакције покреће објављивање по једног налога `RenditionCommand` за сваку резолуцију. `transcoder-service` прима налог, објављује стање `RenditionRunning`, чита изворну датотеку, покреће `FFmpeg` и уписује резултат у дељено складиште. На крају објављује `RenditionSucceeded` или `RenditionFailed`.

Сервис `media-service` примљено стање трајно пресликава у `RUNNING`, `FINISHED` или `ERROR`. После локалне трансакције терминалног исхода објављује обавештење утврђеног облика о појединачној рендицији, а условно и `ALL_COMPLETED` када за видео запис више нема рендиција у стању `PENDING` или `RUNNING`.

`notification-service` свако примљено обавештење уписује у `notification_db`, а затим исти садржај шаље тренутно активним претплатницима. Јавни клијент успоставља овај ток преко `Nginx`, чија конфигурација за ту путању искључује посредно задржавање одговора и његово привремено чување.

Трансакције су локалне по сервису. Упис у базу и накнадно објављивање поруке зато су два узастопна корака, па се целовит ток састоји од више локалних трансакција повезаних порукама преко `RabbitMQ`.

6.5 Типови порука и границе поузданости

Комуникација користи три тематске размене¹⁵ и три трајна реда. Наредбе за израду, догађаји покретања и исхода и терминална обавештења тиме

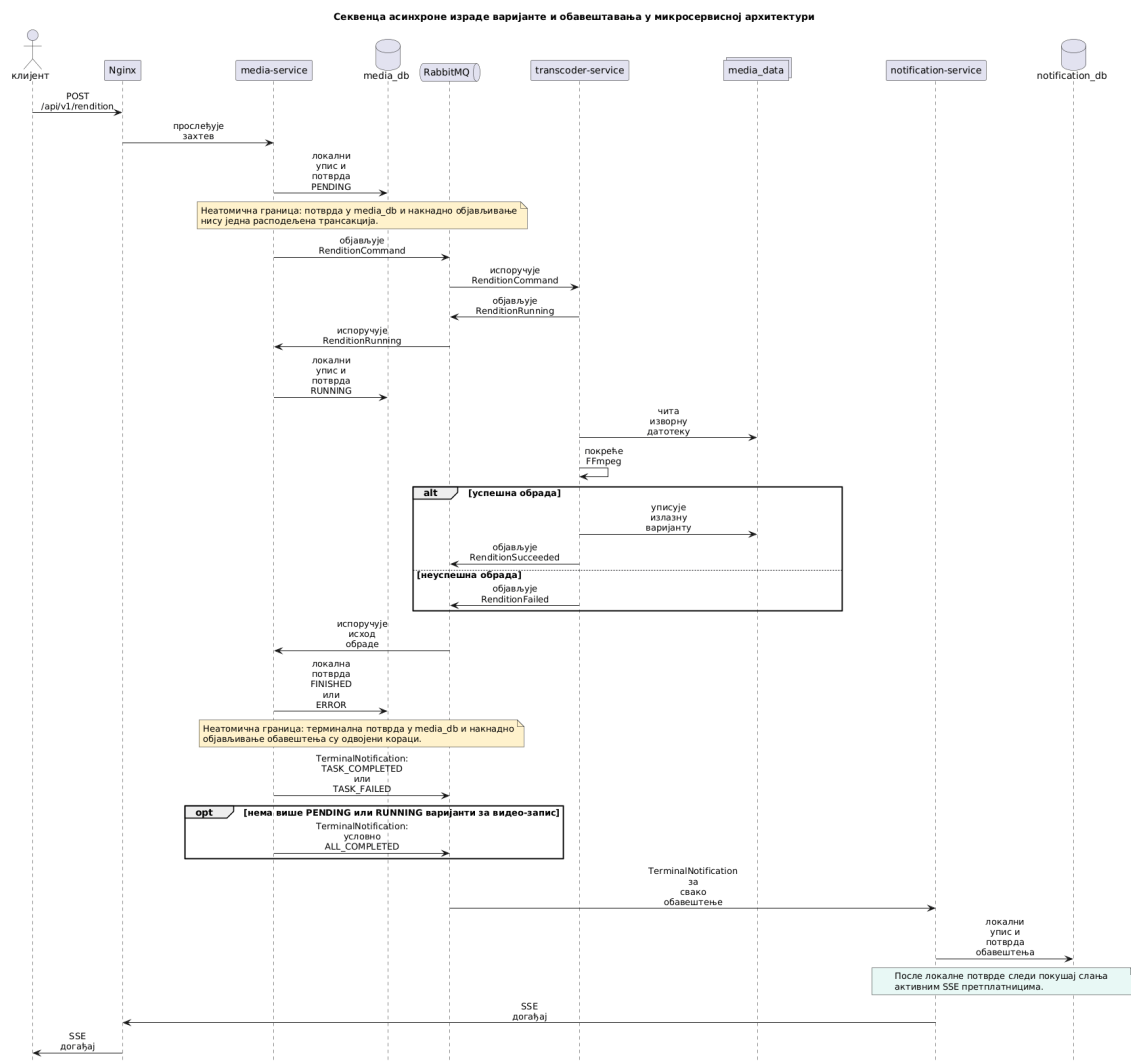
¹⁵Тематска размена у посреднику порука усмерава објављену поруку у ред према кључу и унапред задатим правилима везивања.

прелазе различите процесне границе, док свако трајно стање остаје у надлежном сервису.

Декларација трајних размена и редова не повезује локалну трансакцију базе и касније објављивање поруке у једну недељиву операцију. Имплементација не садржи ни трансакциони излазни ни улазни дневник.¹⁶ Такође нема потврде издавача, свеобухватно уклањање дупликата ни засебан ток за трајно необрадиве поруке. Зато код не затвара ризик губитка публикације после потврде базе, поновљеног рада при поновној испоруци или останка записа у стању `RUNNING` после прекида транскодовања. То су границе поузданости конкретне имплементације. Симетрична проба опоравка није изведена.

Слика 6.2 приказује редослед уписа и слања, јавни повратни ток обавештења кроз `Nginx` и границе локалних трансакција.

¹⁶Ови дневници трајно бележе поруке уз локалну трансакцију или евидентирају њихову обраду, како би накнадно слање и уклањање дупликата могли да се понове после отказа.



Слика 6.2: Секвенца израде рендиције и обавештавања

6.6 Распоредна граница и инструментација

Експериментална ресурсна граница обухвата контејнер мрежног пролаза `gateway`, `media-service`, сваки `transcoder-service`, `notification-service`, `RabbitMQ` и `PostgreSQL` у једној родитељској ресурсној групи. Конфигурација званичног протокола није им задавала уже појединачне границе процесора или меморије, већ је заједнички буџет наметан родитељској групи. Алат `Prometheus` прикупља апликационе метрике са пословних сервиса, док се за званично поређење временски поравнате вредности процесора и радног скупа сабирају над свим наведеним апликационим и инфраструктурним контејнерима. Алат за стварање оптерећења и мерни алати остају изван ресурсне границе.

Конкурентност рада алата `FFmpeg` пресликана је на потрошаче налога у сервису `transcoder-service`. У микросервисној топологији `T1` једна инстанца сервиса за транскодовање користи све четири, односно свих осам радних нити профила. У `T2` две инстанце користе по две, односно по четири нити. Тако промена из `T1` у `T2` уводи додатну процесну и распоредну јединицу, али не мења збирни број радних нити ни заједнички буџет.

6.7 Друга етапа миграције

Табела 6.2 представља нормализовани дневник друге етапе и групише осам промена по семантичком циљу. Промене обухватају одређивање топологије и ресурсне границе, формирање засебних јединица градње и распоређивања, очување јавног програмског интерфејса иза мрежног пролаза и замену локалног покретања типовима и правилима размене порука. Оне такође обухватају раздвајање власништва над подацима, задржавање обраде датотека преко заједничког тома, укључивање микросервисне имплементације у мерни поступак и израду спецификација, дијаграма и записа провере. Тиме су обухваћене све категорије захваћених артефаката: одлуке, градња и распоређивање, јавни програмски интерфејс, комуникација и трансакције, подаци и шема, складиште, мерење и документација.

Семантички циљ	Категорија	Пратећи артефакти
Утврђивање топологије, власништва и ресурсне границе	одлука	спецификација имплементације
Засебне јединице градње и распоређивања	градња и распоређивање	сервисни модули, описи градње и контејнерске слике
Очување јавног програмског интерфејса иза мрежног пролаза	јавни програмски интерфејс	спецификација јавног програмског интерфејса, контролери и конфигурација мрежног пролаза
Замена локалног покретања типовима и правилима размене порука	комуникација и трансакције	спецификација размене, типови порука, произвођачи и потрошачи
Раздвајање власништва над трајним подацима	подаци и шема	<code>media_db</code> , <code>notification_db</code> , корисници и скрипте иницијализације
Очување локалне обраде преко дељеног тома	складиште	адаптери датотека и том <code>media_data</code>
Укључивање микросервисне имплементације у заједничко мерење	мерење	покретач, извоз исхода, манифест улаза и распоред
Израда спецификација, дијаграма и записа провере	документација	спецификације, дијаграми и записи провере

Табела 6.2: Нормализоване семантичке јединице друге етапе миграције

Ових осам промена показује да процесна декомпозиција није била само премештање класа, већ је увела нова правила размене порука, приступне налоге и права над подацима, јединице распоређивања и мерне границе. Њихов број није мера времена, труда или сложености. За прву етапу не постоји дневник нормализован по истом правилу: она је реконструисана ретроспективно и описана квалитативно. Зато се бројеви двеју етапа не пореде.

Глава 7

Резултати и дискусија

Свих 80 унапред планираних покретања било је технички важеће, а у сваком је завршено свих 36 рендиција, без пословних грешака и неуспелих транспортних захтева.

Из резултата се издвајају четири налаза. Већи ресурсни профил прати већа пропусност, краће трајање и нижа латенција успешне групе у свим експерименталним варијантама. У односу на монолит, средња пропусност модуларног монолита кретала се од 9,3% ниже до 2,3% више, трајање од 3,6% краће до 7,9% дуже, а латенција успешне групе од 3,4% ниже до 7,6% више. Ови распони не успостављају практичну еквиваленцију. Микросервисне топологије углавном имају нижу пропусност, дуже трајање и већи средњи радни скуп. T2 није доследно бољи од T1 када укупан капацитет остаје исти.

На графиконима апсолутних метрика круг представља једно покретање, квадрат средњу вредност, а линија или трака 95% интервал поверења. Интервал показује неизвесност процене, али није тест статистичке значајности. Графикон релативне пропусности приказује количник средњих вредности, при чему је монолит постављен на 100%. Табеле приказују средњу вредност и узорачку стандардну девијацију.

7.1 Пропусност и трајање оптерећења

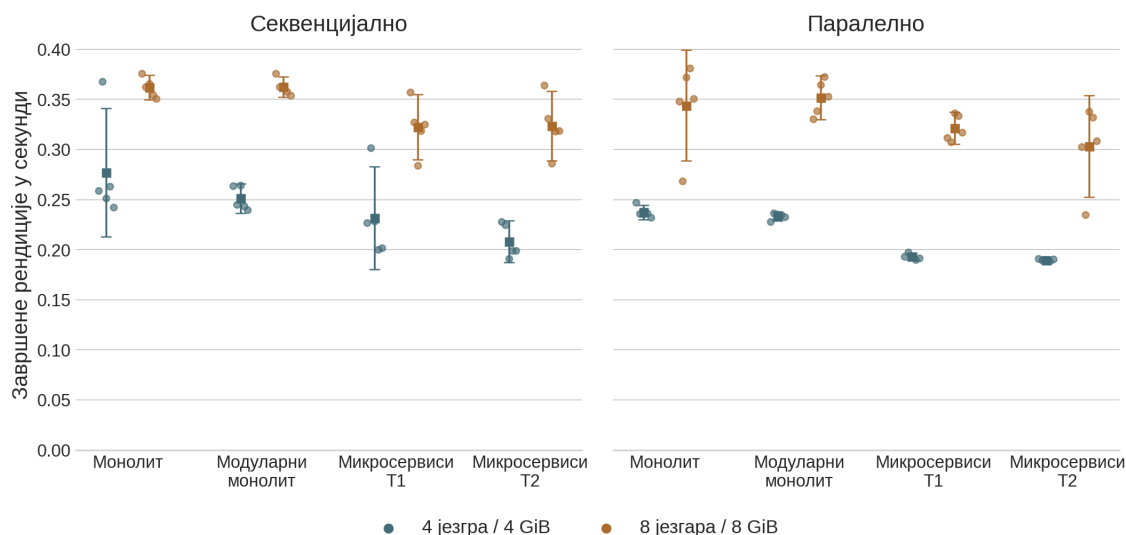
Према дефиницијама из Табеле 3.2 у Поглављу 3, пропусност показује колико се посла заврши у секунди, а трајање колико је било потребно да се заврши фиксно оптерећење од 36 рендиција. Зато су ове две мере у сваком покретању обрнуто повезане. Када је обим завршеног рада непромењен, већа

пропусност значи краће трајање. Табела 7.1 зато приказује пропусност, а не понавља њен инверзни приказ кроз трајање.¹⁷

Експериментална варијанта	Начин	4/4 средња вредност ±SD	8/8 средња вредност ±SD
Монолит	сек.	0,2769 ± 0,0516	0,3621 ± 0,0098
Монолит	пар.	0,2372 ± 0,0058	0,3440 ± 0,0445
Модуларни монолит	сек.	0,2512 ± 0,0119	0,3626 ± 0,0082
Модуларни монолит	пар.	0,2334 ± 0,0035	0,3519 ± 0,0175
Микросервиси Т1	сек.	0,2317 ± 0,0413	0,3225 ± 0,0261
Микросервиси Т1	пар.	0,1930 ± 0,0029	0,3214 ± 0,0130
Микросервиси Т2	сек.	0,2083 ± 0,0168	0,3237 ± 0,0280
Микросервиси Т2	пар.	0,1897 ± 0,0011	0,3031 ± 0,0410

Табела 7.1: Пропусност у завршеним рендицијама у секунди по пет покретања ($n = 5$). Сваки резултат је приказан као средња вредност ± узорачка стандардна девијација (SD). Ознаке 4/4 и 8/8 представљају број процесорских језгара и гибџајта меморије, а сек. и пар. означавају начин оптерећења

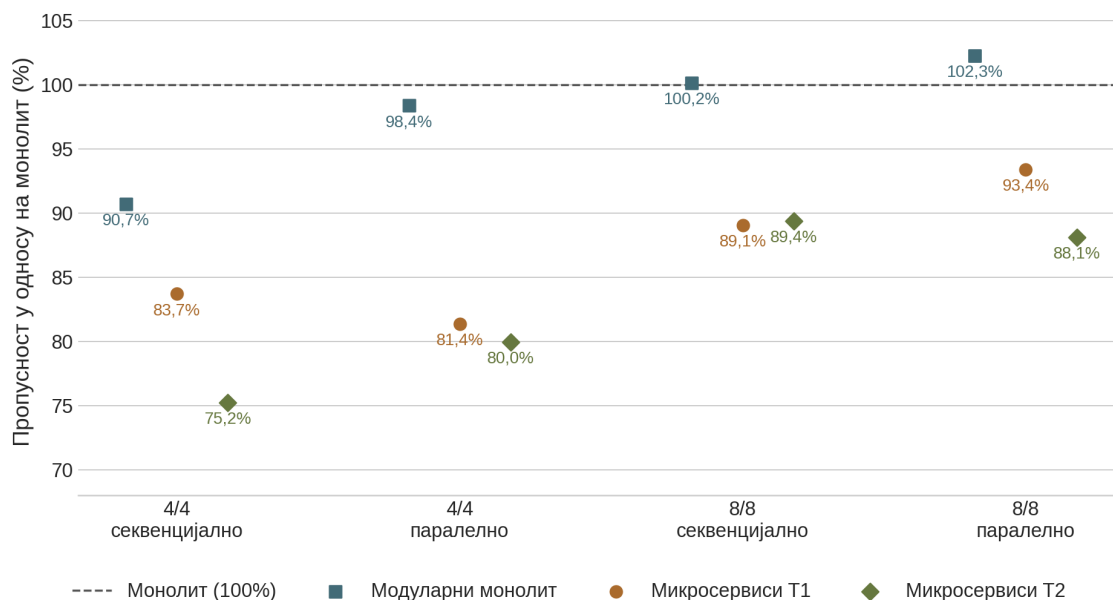
Већи профил прати већу пропусност и краће трајање у свакој експерименталној варијанти и при оба начина оптерећења.



Слика 7.1: Пословна пропусност по експерименталној варијанти, ресурсном профилу и начину оптерећења

¹⁷Релативне разлике у трајању израчунате су поређењем аритметичких средњих вредности трајања пет покретања унутар истог ресурсног профила и начина оптерећења, а не из реципрчних вредности приказаних средњих пропусности.

На Слици 7.1 већи профил је изнад мањег у свакој експерименталној варијанти. Појединачне тачке истовремено показују да резултати нису једнаки у сваком понављању.



Слика 7.2: Средња пропусност у односу на монолит (монолит = 100%) по профили и начину оптерећења. Поређење је описно

Релативни приказ на Слици 7.2 олакшава поређење конкретних имплементација и топологија. У односу на монолит, средња пропусност модуларног монолита била је 9,3% нижа, 1,6% нижа, готово једнака и 2,3% виша, редом у условима 4/4 секвенцијално, 4/4 паралелно, 8/8 секвенцијално и 8/8 паралелно. Праг практичне еквиваленције није био унапред одређен. Микросервисне топологије су испод монолита. Односи пропусности T2/T1 износили су 0,899, 0,983, 1,004 и 0,943 у четири испитана услова. Пошто се неки интервали преклапају, графиконе треба читати као образац у овом узорку, а не као коначан ранг архитектуралних образаца изван испитаних имплементација.

7.2 Латенција успешне групе

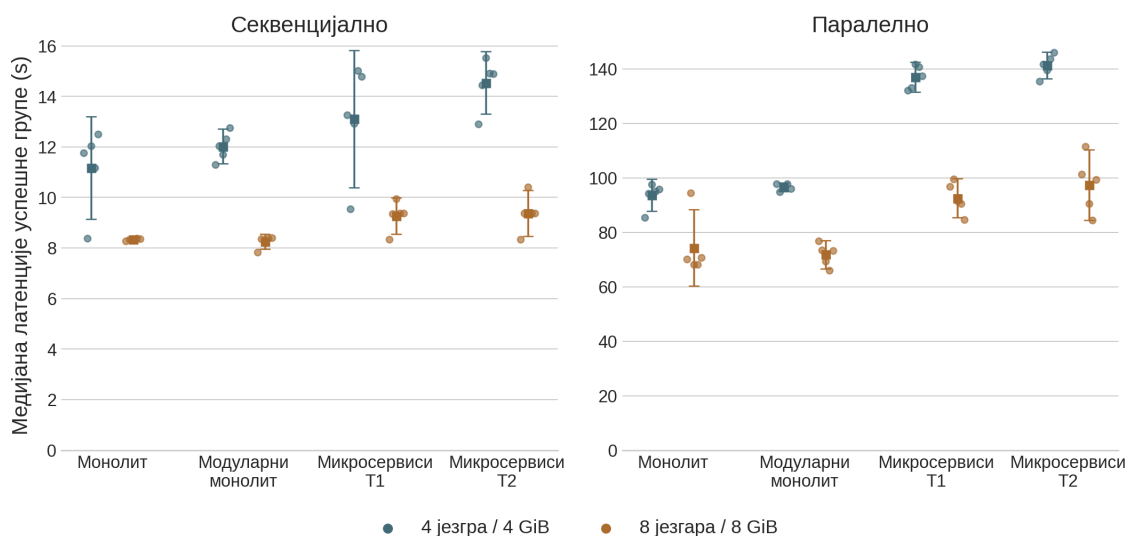
Латенција успешне групе, изражена у секундама, јесте време од прихватања једног захтева за израду до завршетка све три планиране

рендиције. Код секвенцијалног оптерећења она је блиска трајању обраде једне групе. Код паралелног оптерећења укључује и чекање у реду, јер се више захтева подноси истовремено, а деле исти капацитет. За свако покретање прво је израчуната медијана латенције успешних група, а Табела 7.2 приказује средњу вредност пет таквих медијана.

Експериментална варијанта	4/4 сек.	4/4 пар.	8/8 сек.	8/8 пар.
Монолит	11,17 $\pm$ 1,63	93,61 $\pm$ 4,73	8,34 $\pm$ 0,04	74,27 $\pm$ 11,31
Модуларни монолит	12,02 $\pm$ 0,55	96,50 $\pm$ 1,26	8,25 $\pm$ 0,23	71,78 $\pm$ 4,19
Микросервиси Т1	13,11 $\pm$ 2,19	136,97 $\pm$ 4,36	9,27 $\pm$ 0,58	92,55 $\pm$ 5,83
Микросервиси Т2	14,54 $\pm$ 1,00	141,29 $\pm$ 4,02	9,37 $\pm$ 0,73	97,38 $\pm$ 10,44

Табела 7.2: Средња вредност медијана латенције успешних група по покретању $\pm$ SD, у секундама ($n = 5$). Ознака SD представља узорачку стандардну девијацију, а 4/4 и 8/8 број процесорских језгара и гибибајта меморије

Већи профил је пратио нижу вредност у свих осам поређења. Сва понављања приказује Слика 7.3.



Слика 7.3: Медијана латенције успешне групе од три рендиције. Панели имају различите распоне вертикалне осе

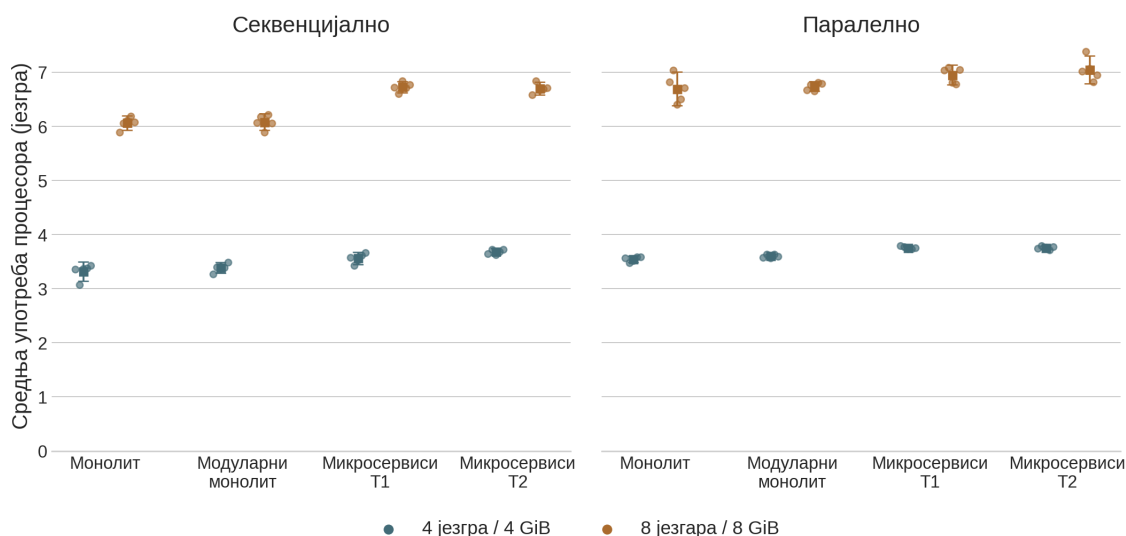
Без обзира на експерименталну варијанту, профил и начин оптерећења, латенција рендиције по резолуцији има исти редослед: SD_360 има најнижу, затим следи SD_480, а HD_720 има највишу вредност. Ово је секундарна дијагностичка мера у секундама, док је латенција успешне групе главна

мера за поређење експерименталних варијанти. Исти редослед остаје и при паралелном оптерећењу, где измерено време укључује и чекање у реду.

7.3 Употреба процесора и меморије

Графикони употребе процесора и меморије дају контекст резултатима мерења перформанси, али не доказују њихов узрок. За свако покретање приказана средња вредност обухвата његово сопствено мерно трајање, а групна средња вредност сажима пет покретања. Зато ове мере описују заузетост ресурса током истог сценарија. Обе мере су агрегати свих чланова ресурсне границе, односно свих делова распоређене варијанте, а не само пословних сервиса. Код монолитних варијанти у агрегат улази и база података, а код микросервиса и посредник порука са својим редовима.

Средњу употребу процесора са појединачним понављањима приказује Слика 7.4.

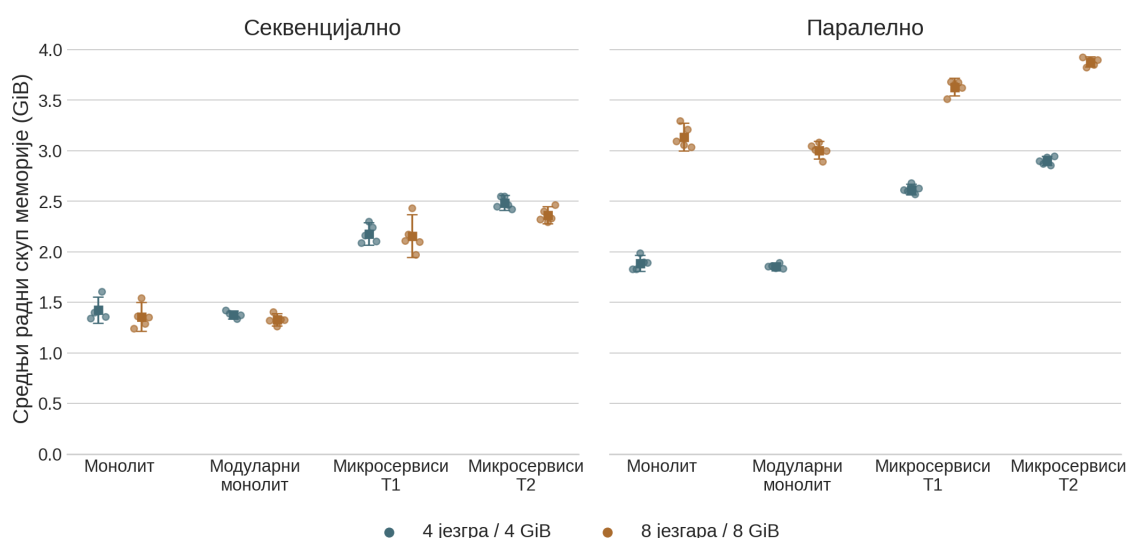


Слика 7.4: Средња вредност агрегата употребе процесора свих чланова ресурсне границе

Средња употреба процесора расте са већим профилем у свим експерименталним варијантама. Две микросервисне топологије у два профила и при два начина оптерећења дају осам микросервисних услова. У сваком од њих средња заузетост била је виша од одговарајуће вредности обе монолитне варијанте, што даје 16 парних поређења. Виша заузетост прати

додатне процесе и посредовање порукама, али сама по себи не издваја чист архитектурални трошак, јер се исти рад завршава уз другачију пропусност и трајање.

Средњи радни скуп микросервисних топологија приказан је на Слици 7.5.



Слика 7.5: Средња вредност агрегата радних скупова свих чланова ресурсне границе

Он је већи од обе монолитне варијанте у свакој комбинацији профила и начина оптерећења. То одговара распоређивању већег броја процеса, али не доказује да је оно једини узрок. Највећи радни скуп остаје дијагностичка мера, док се за поређење користи средња вредност.

7.4 Шта резултати показују

Полазећи од критеријума из Поглавља 2, Табела 7.3 сажима главне налазе и границе њиховог тумачења.

Шта је уочено	Како се може разумети	На шта треба пазити
Већи профил прати већу пропусност, краће трајање и нижу латенцију	Овај образац је уочен у свим испитаним експерименталним варијантама	Процесор, меморија и број нити мењају се заједно
Средња пропусност модуларног монолита била је од 9,3% нижа до 2,3% виша од монолита	Разлике између ове две монолитне варијанте зависиле су од испитаног услова	Праг практичне еквиваленције није одређен и еквиваленција није испитивана
Испитане микросервисне топологије углавном имају нижу пропусност, дуже трајање, већи радни скуп и вишу употребу процесора	Одвојено распоређивање није било праћено већом пропусношћу при истом буџету	Узрок разлике није издвојен
T2 није доследно бољи од T1	Већи број инстанци сервиса није довољан ако се укупан капацитет не повећа	Важи за фиксни буџет и испитана оптерећења

Табела 7.3: Преглед главних налаза и ограничења

За ИП1 резултати показују да већи профил прати боље вредности главних мера перформанси. Средња пропусност модуларног монолита била је од 9,3% нижа до 2,3% виша од пропусности монолита, зависно од профила и начина оптерећења, без унапред одређеног прага практичне еквиваленције. Микросервисне топологије обично имају нижу пропусност, дуже трајање и већи радни скуп, нарочито у профилу 4/4. T2 није доследно бољи од T1. Употреба процесора и меморије и латенција рендиције по резолуцији допуњују овај одговор као дијагностичке мере. Одговор важи за један систем, заједничку ресурсну границу, два профила, два начина оптерећења и испитане топологије.

За ИП2 докази су квалитативни и показују разлике у организацији кода, предусловима и топологији распоређивања, као и видљиве механизме отказа и границе поузданости.

За ИП3 врсте промена су квалитативно описане у обе етапе.

Глава 8

Закључак

Рад је упоредио конкретне имплементације монолита, модуларног монолита и микросервиса као три архитектуралне варијанте истог система за обраду видео записа. Поређење је обухватило архитектуралне границе, јавни асинхрони ток, распоређивање, употребу ресурса и документоване промене током две етапе миграције. Примарни квантитативни доказ је потпуна матрица са 80 технички важећих покретања, без замена: четири експерименталне варијанте, два ресурсна профила, два начина оптерећења и пет независних понављања.

8.1 Одговори на истраживачка питања

ИП1. У свих 80 покретања завршено је свих 36 планираних рендиција без забележених пословних грешака и неуспелих транспортних захтева. Према Табели 7.1 у Поглављу 7, средња пропусност била је виша при већем профилу у свакој експерименталној варијанти и при оба начина оптерећења. Количници 8/8 према 4/4 кретали су се у распону од 1,31 до 1,66.

У зависности од профила и начина оптерећења, средња пропусност модуларног монолита била је од 9,3% нижа до 2,3% виша од пропусности монолита, трајање од 3,6% краће до 7,9% дуже, а латенција успешне групе од 3,4% ниже до 7,6% више. Пошто праг практичне еквиваленције није био унапред одређен, ови распони не доказују практичну једнакост варијанти. Микросервисне топологије обично су имале нижу пропусност, дуже трајање и већи средњи радни скуп, нарочито у профилу 4/4. Уз то су имале и вишу средњу вредност агрегата употребе процесора свих чланова ресурсне границе,

што се не може назвати мањом ефикасношћу јер су трајања и пропусности различити.

Односи пропусности $T2/T1$ износили су 0,899 и 0,983 у профилу 4/4, а 1,004 и 0,943 у профилу 8/8, за секвенцијално и паралелно оптерећење. Додавање друге инстанце сервиса за транскодовање уз фиксни укупан буџет и агрегатни број радних нити стога није имало доследан смер. Према Табели 7.2, већи профил је у свих осам одговарајућих поређења смањио средњу вредност медијана латенције успешне групе по покретању, изражену у секундама. Латенција рендиције по резолуцији, такође изражена у секундама, задржана је као секундарна дијагностичка мера.

ИП2. Испитана монолитна имплементација има једну пословну јединицу распоређивања и границе пакета које нису машински спроведене. Испитани модуларни монолит задржава један процес, али уводи именовани међумодулски интерфејс и дефинисан поступак провере. Испитана микросервисна имплементација уводи три пословна процеса, типове и правила размене порука, засебне приступне налоге и логичке базе, али задржава заједнички том и процес базе. Код не затвара све ризике између потврде локалне трансакције и објављивања поруке.

ИП3. Прва етапа реконструисана је ретроспективно и у њој су издвојене три ненормализоване врсте промена, и то издвајање заједничких основних класа, увођење објављеног интерфејса за догађаје и додавање поступка за проверу и приказ модулске структуре.

Друга етапа има осам нормализованих семантичких јединица из Табеле 6.2. Забележена је по једна јединица у свакој од осам категорија. Категорије су одлука, градња и распоређивање, јавни програмски интерфејс, комуникација и трансакције, подаци и шема, складиште, мерење и документација. Ова расподела описује врсте захваћених промена, а не време, трошак, сложеност или уложени рад. Пошто прва етапа није евидентирана истим правилом, кардиналности двеју етапа не пореде се.

8.2 Практично значење и будући рад

За испитани систем разлике средњих вредности монолита и модуларног монолита остале су у наведеним распонима, уз експлицитнији међумодулски интерфејс и без расподељеног тока. Испитана микросервисна

имплементација уводи засебне распоредне границе и границе података. Њена примена, међутим, захтева управљање порукама, већим бројем процеса и делимичним отказима. Само повећање броја инстанци сервиса за транскодовање, без додавања укупног капацитета, у овом случају није обезбедило доследан прираст пропусности. Закључци важе за испитани систем, оптерећење и ресурсне профиле и не успостављају универзални поредак архитектуралних образаца.

Будући рад треба засебно да испита друге нивое оптерећења, веће и разноврсније улазе, додавање стварног капацитета уз хоризонталне топологије и симетричне пробе поузданости. Већи број понављања и унапред планирани тестови били би потребни за инференцијалне тврдње.

Библиографија

- [1] Andy Georges, Dries Buytaert, and Lieven Eeckhout. Statistically Rigorous Java Performance Evaluation. In *Proceedings of the 22nd Annual ACM SIGPLAN Conference on Object-Oriented Programming Systems, Languages and Applications*, pages 57–76, 2007.
- [2] James Lewis and Martin Fowler. Microservices. <https://martinfowler.com/articles/microservices.html>, 2014. Приступљено 11. 7. 2026.
- [3] Len Bass, Paul Clements, and Rick Kazman. *Software Architecture in Practice*. SEI Series in Software Engineering. Addison-Wesley Professional, 3 edition, 2012.
- [4] Leonardo Passos, Ricardo Terra, Marco Tulio Valente, Renato Diniz, and Nabor das Chagas Mendonca. Static Architecture-Conformance Checking: An Illustrative Overview. *IEEE Software*, 27(5):82–89, 2010.
- [5] Martin Fowler. Monolith First. <https://martinfowler.com/bliki/MonolithFirst.html>, 2015. Приступљено 15. 6. 2026.
- [6] Nicola Dragoni, Saverio Giallorenzo, Alberto Lluch Lafuente, Manuel Mazara, Fabrizio Montesi, Ruslan Mustafin, and Larisa Safina. Microservices: Yesterday, Today, and Tomorrow. In *Present and Ulterior Software Engineering*, pages 195–216. Springer, Cham, 2017.
- [7] Raj Jain. *The Art of Computer Systems Performance Analysis: Techniques for Experimental Design, Measurement, Simulation, and Modeling*. Wiley Professional Computing. John Wiley & Sons, 1991.
- [8] Ruoyu Su and Xiaozhou Li. Modular Monolith: Is This the Trend in Software Architecture? In *Proceedings of the 1st International Workshop on New*

- Trends in Software Architecture*, pages 10–13, New York, NY, USA, 2024. Association for Computing Machinery.
- [9] Samuel Kounev, Klaus-Dieter Lange, and Jóakim von Kistowski. *Systems Benchmarking: For Scientists and Engineers*. Springer, 2020.
- [10] Spring. Spring Modulith Reference Documentation: Fundamentals and Verifying Application Module Structure. <https://docs.spring.io/spring-modulith/reference/>, 2026. Приступљено 11. 7. 2026.